

```

# =====
# GPA Calculator with Transcript
# =====

class Course:
    def __init__(self, name, credits):
        self.name = name
        self.credits = credits

class Grade:
    grade_points = {
        "A": 4.0,
        "B": 3.0,
        "C": 2.0,
        "D": 1.0,
        "F": 0.0
    }

    def __init__(self, grade_symbol):
        if grade_symbol not in Grade.grade_points:
            raise ValueError(f"Invalid grade: {grade_symbol}")
        self.symbol = grade_symbol
        self.points = Grade.grade_points[grade_symbol]

class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.courses = [] # list of (Course, Grade)

    def add_course(self, course, grade_symbol):
        grade = Grade(grade_symbol)
        self.courses.append((course, grade))

    def calculate_gpa(self):
        if not self.courses:
            return 0.0
        total_points, total_credits = 0, 0
        for course, grade in self.courses:
            total_points += grade.points * course.credits
            total_credits += course.credits
        return round(total_points / total_credits, 2)

    def generate_transcript(self):
        for course, grade in self.courses:
            yield course.name, grade.symbol, course.credits

# =====
# Unit Tests (Corrected)
# =====
import unittest

class TestGPA_Calculation(unittest.TestCase):
    def setUp(self):
        self.student = Student(1010, "Rohit Sharma")
        self.student.add_course(Course("Data Science", 3.0), "B") # 9
        self.student.add_course(Course("Machine Learning", 4.0), "A") # 16
        self.student.add_course(Course("Cyber Security", 3.0), "C") # 6
        # Total points = 31, Total credits = 10 → GPA = 3.1

    def test_gpa_calculation(self):
        # ✅ Correct expected GPA
        self.assertEqual(self.student.calculate_gpa(), 3.1)

# =====
# Run Example + Clean Test Output
# =====
if __name__ == "__main__":
    student_1 = Student(1010, "Rohit Sharma")
    student_1.add_course(Course("Data Science", 3.0), "B")
    student_1.add_course(Course("Machine Learning", 4.0), "A")
    student_1.add_course(Course("Cyber Security", 3.0), "C")

```

```
gpa = student_1.calculate_gpa()

print("\n--- Example System Usage ---")
print(f"Student: {student_1.name} (ID: {student_1.student_id})")
print("-" * 30)
print(f"Calculated GPA: {gpa}")
print("Transcript:")
for name, grade, credits in student_1.generate_transcript():
    print(f"- {name} ({credits:.1f} cr): {grade}")

print("\nRunning Unit Tests...\n")

# Run only the intended test class
suite = unittest.TestLoader().loadTestsFromTestCase(TestGPA_Calculation)
unittest.TextTestRunner(verbosity=1).run(suite)
```

```
.
-----
Ran 1 test in 0.001s

OK

--- Example System Usage ---
Student: Rohit Sharma (ID: 1010)
-----
Calculated GPA: 3.1
Transcript:
- Data Science (3.0 cr): B
- Machine Learning (4.0 cr): A
- Cyber Security (3.0 cr): C

Running Unit Tests...
```